# Progee v2: An Evidence-First, State-Machine-Governed System for AI-Assisted Software Engineering

TBD

January 2026

**Abstract**

AI-assisted software engineering can reduce implementation time but also introduces new failure modes: unverifiable claims, weak test oracles, and opaque decision-making that makes audits difficult. We present **Progee v2**, a prototype system that governs AI-assisted development using (1) an explicit task state machine that gates progress through verification steps, and (2) evidence-first persistence of artifacts and checks in a database. The system is implemented in Delphi (FMX) with PostgreSQL storage and DUnitX tests. We release reproducible evaluation artifacts based on end-to-end (E2E) tests that generate machine-readable XML reports, and we discuss current limitations, including an incomplete mutation-testing executor.

Paper type: System / Tool Draft date: 2026-01-26

## 1 1. Introduction

### 1.1 1.1 One-sentence positioning (PAPER-001)

**Progee v2 is a governance-oriented AI-assisted development system that makes each task's progress, verification, and failure context auditable by design via an explicit state machine and evidence-first storage.**

### 1.2 1.2 Contributions (PAPER-001)

This paper's contributions are:

1. **Evidence-first artifact persistence for auditability**, including redaction support and bounded evidence snippets for UI/prompt contexts.

- Implementation: `CtrlEvidence.pas`, `sql/16_context_evidence_objects.sql`

1. **A task state machine that enforces gated verification steps**, including an explicit rework loop with persisted failure history (`tasks.failure_context`) that can be summarized back into prompts.

- Implementation: `CtrlTaskStateMachine.pas`, `CtrlWorkshopThread.pas`, `sql/17_tasks_failure_context`

1. **A reproducibility package for machine-readable evaluation evidence**, centered on DUnitX XML outputs for E2E tests and a replication guide.

- Implementation: `scripts/run_e2e_tests.ps1`, `docs/Replication.md`

1. **A tested model escalation policy module ("model racing")** that maps rework counts to model levels (policy), intended to be wired into the execution path.

- Implementation: `CtrlModelRacing.pas`, `tests/IntegrationModelRacing.pas`, `sql/05_seed_data.sql`

1. **A prototype mutation-testing gate integrated into the governance pipeline**, with persisted reports and thresholds, while explicitly acknowledging the executor is not yet production-grade.

- Implementation: `sql/11_mutation_testing.sql`, `CtrlMutationEngine.pas`, `CtrlMutationOperators.pas` (contains TODO/placeholder logic)

### 1.3 1.3 Contribution-to-evidence mapping (PAPER-004)

For each contribution, we provide a minimal verification path (commands and artifacts):

- **C1 Evidence objects**: run E2E tests (Section 6.2) to populate `progee_app_self.context_evidence_objec` then export `eval/evidence_counts.csv` via `scripts/export_eval_metrics.ps1` or query `scripts/eval_queries.sql` (section 5/6). Optionally inspect individual rows (type, sha256, redacted).
- **C2 State machine + rework failure history**: run E2E tests, then query `progee_app_self.tasks.status`, `rework_count`, and `failure_context` (see `scripts/eval_queries.sql`, section 4). Failure history is summarized into prompts by `CtrlWorkshopThread.pas`.
- **C3 Reproducibility package**: run `scripts/run_e2e_tests.ps1` → `bin/dunitx-results.xml` (machine-readable evidence).
- **C4 Model racing policy**: run `tests/IntegrationModelRacing.pas` (DUnitX) and verify default policy keys in `progee_core.system_config` (seeded by `sql/05_seed_data.sql`).
- **C5 Mutation-testing gate (prototype)**: run E2E tests, then query `progee_app_self.mutation_test_res` / `mutation_details`; export summaries via `scripts/export_eval_metrics.ps1` (`mutation_score_summary` `mutation_runs_by_status.csv`).

### 1.4 1.4 Scope

This paper is a **system/tool** paper. We emphasize implemented mechanisms and reproducible artifacts over broad causal claims about defect rates.

## 2 2. Motivation and Problem Statement

### 2.1 2.1 Why governance, not only generation

LLM-based code generation can be fast, but correctness and safety depend on:

- explicit, enforceable verification gates;
- auditable evidence of what was checked;
- robust handling of failures and rework.

### 2.2 2.2 Target user workflow

Progee v2 targets workflows where:

- tasks are decomposed into verifiable steps;
- the system must provide evidence for acceptance decisions;
- regression is caught by automated tests and (optionally) mutation testing.

## 3 3. System Overview

### 3.1 3.1 Core entities and persistence

At a high level, Progee v2 persists:

- tasks and their states (state machine);
- artifacts and evidence objects (e.g., test reports, logs, code snippets);
- model selection decisions;
- verification outcomes.

Key design principle: **the database is the source of truth**, so the UI and prompts can be derived from persisted evidence rather than ephemeral chat history.

### 3.2 3.2 State machine and gates

The task lifecycle is governed by explicit states (including rework and verification stages).

- Implementation anchor: `CtrlTaskStateMachine.pas`

### 3.3 3.3 Evidence-first collection

Evidence objects are stored with controlled size (snippets) and optional redaction.

- Implementation anchor: `CtrlEvidence.pas`

## 4 4. Governance Mechanisms

### 4.1 4.1 Rework loop with recorded failure context

When a task fails a verification gate, Progee records failure context and requests rework.

- Implementation anchor: `CtrlTaskStateMachine.pas` (failure_context JSON)

### 4.2 4.2 Model escalation policy ("model racing")

Progee v2 includes a policy module that maps rework counts to model tiers (e.g., level-1 → level-2 at 2 reworks, level-2 → level-3 at 4 reworks). This module is validated by integration tests; wiring it into the AI adapter / per-task execution path is future work.

- Implementation anchor: `CtrlModelRacing.pas`, `tests/IntegrationModelRacing.pas`

### 4.3 4.3 Verification pipeline (tests; optional mutation gate)

The workshop pipeline integrates generation and verification steps and persists results.

- Implementation anchor: `CtrlWorkshopThread.pas`

Mutation testing is represented as a gate with schema and plumbing, but the executor is currently incomplete.

- Implementation anchor: `CtrlMutationEngine.pas`, `CtrlMutationOperators.pas`

## 5 5. Implementation

### 5.1 5.1 Technology stack

- Delphi / FireMonkey (FMX) application
- PostgreSQL persistence
- DUnitX for testing

### 5.2 5.2 Database schema

Key schema files:

- Evidence objects: `sql/16_context_evidence_objects.sql`
- Mutation testing: `sql/11_mutation_testing.sql`

### 5.3 5.3 Test harness

The repo provides a DUnitX test runner:

- `bin/ProgeeTests.exe`

## 6 6. Evaluation (reproducible evidence-first)

### 6.1 6.1 What we evaluate

We focus on **reproducibility of verification evidence**:

- E2E test suite execution
- Machine-readable XML outputs

We avoid claiming defect-rate improvements unless fully supported by reproducible artifacts.

### 6.2 6.2 How to reproduce

See `docs/Replication.md`. Minimal steps:

1. Configure a dedicated PostgreSQL database whose name contains `_test`.
2. Set environment variables including `PROGEE_ALLOW_DESTRUCTIVE_TESTS=1`.
3. Run:

```
.\scripts\run_e2e_tests.ps1
```

Expected output:

- `bin/dunitx-results.xml`

### 6.3 6.3 Optional metrics extracted from DB

For plots/tables, export minimal metrics from DB:

- number of reworks per task
- evidence object counts and size distribution
- mutation score (if available)
- AI-call volume and token usage (if logs are enabled)

We provide:

- `scripts/eval_queries.sql` (ad-hoc queries)
- `scripts/export_eval_metrics.ps1` → outputs CSVs to `eval/`

### 6.4 6.4 Evidence-first end-to-end example (ENG-PAPER-001)

We provide a minimal, end-to-end evidence chain that can be reproduced from the system:

1. **Task → evidence_ref**: during `quality_check` or `mutation_test`, Progee persists a validation or mutation report into `progee_app_self.context_evidence_objects` and stores its `id` as `evidence_ref` in `tasks.failure_context`.
2. **evidence_ref → snippet**: the UI's Evidence tab uses bounded snippet retrieval (`tail` / `grep`) via `CtrlEvidence.pas` to retrieve a safe excerpt of the referenced evidence object.
3. **snippet → rework prompt**: on rework, `CtrlWorkshopThread.BuildFailureContextForPrompt` injects the failure summary plus the snippet back into the AI prompt to avoid repeating past mistakes.
4. **audit record**: each status transition and mutation-gate decision is recorded in `progee_app_self.audit_logs`

Minimal reproduction (no manual UI):

- Run E2E tests (`scripts/run_e2e_tests.ps1`) to populate tasks, failure_context, and evidence objects.
- Query `context_evidence_objects` and `tasks.failure_context` (see `scripts/eval_queries.sql`).
- Verify audit entries in `audit_logs` for task status changes and mutation-gate decisions.

# 7 7. Limitations & Threats to Validity (PAPER-005)

- **Mutation testing executor is not production-grade yet**: `CtrlMutationOperators.pas` contains placeholder/TODO logic for running tests and compile checks; the paper must not claim strong empirical benefits from mutation testing at this stage.
- **Model escalation is currently a policy module**: model racing logic is implemented and tested, but the current AI provider/model selection is primarily global-config driven (via `CtrlAIAdapter.pas`). Wiring escalation into per-task execution is future work.
- **Configuration split**: some thresholds and settings are stored in different tables/schemas (e.g., `progee_core.system_config` vs app-level tables). Replication should explicitly document which keys are required.
- **External validity**: E2E tests in this repo demonstrate the system on the included workflows; applicability to other languages/toolchains requires adapters.
- **Dependence on test quality**: like all testing-based verification, the strength of evidence depends on the quality of written tests.

# 8 8. Related Work (PAPER-003)

**Specifications / contracts.** Formal specification and correctness reasoning date back to Hoare's axiomatic basis for program correctness, while Design by Contract makes pre/postconditions a first-class discipline for software construction. JML provides a practical specification language and tool ecosystem for Java that operationalizes contract-like design by integrating runtime checking and verification tools.

**Traceability, auditability, provenance.** Requirements traceability has long been recognized as a core engineering problem, with classic analyses distinguishing pre-RS and post-RS traceability and modern books consolidating practice across the lifecycle. Recent work on traceability in the wild highlights real-world gaps in link completeness. For provenance and supply-chain auditability, the W3C PROV family formalizes provenance representations, while SLSA provides a practical, staged framework for build provenance and verification.

**Mutation testing.** Mutation testing has an extensive body of theory, tools, and empirical studies, including major surveys and practical systems (e.g., µJava) that operationalize mutation operators and scoring for test adequacy.

**AI-assisted software engineering & agentic verification.** Code LMs (Codex, AlphaCode) show strong synthesis ability but still require robust evaluation and governance. SWE-bench and SWE-agent provide repository-scale benchmarks and agent interfaces. A growing body of agentic and self-critique methods (ReAct, Reflexion, Self-Refine) explores how models can plan, act, and refine; alignment-oriented ideas like Constitutional AI and Debate provide mechanisms for structured critique or oversight. Progee v2 complements these lines with an explicit, auditable state machine and evidence-first persistence.

**References (selected)**


1. C. A. R. Hoare. *An Axiomatic Basis for Computer Programming.* Communications of the ACM, 1969.
2. B. Meyer. *Applying Design by Contract.* IEEE Computer, 25(10), 1992.

3. L. Burdy et al. *An Overview of JML Tools and Applications.* Software Tools for Technology Transfer, 7(3), 2005.

4. O. Gotel & A. Finkelstein. *An Analysis of the Requirements Traceability Problem.* ICRE/RE, 1994.

5. J. Cleland-Huang, O. Gotel, A. Zisman (eds.). *Software and Systems Traceability.* Springer, 2012.

6. M. Rath et al. *Traceability in the Wild: Automatically Augmenting Incomplete Trace Links.* 2018.

7. W3C. *PROV-N: The Provenance Notation.* W3C Recommendation, 2013.

8. SLSA Community. *SLSA v1.1 Specification* (Approved). 2025.

9. B. Kitchenham, T. Dybå, M. Jørgensen. *Evidence-Based Software Engineering.* ICSE, 2004.

10. Y. Jia & M. Harman. *An Analysis and Survey of the Development of Mutation Testing.* IEEE TSE, 37(5), 2011.

11. M. Papadakis et al. *Mutation Testing Advances: An Analysis and Survey.* Advances in Computers, 2019.

12. Y.-S. Ma, J. Offutt, Y.-R. Kwon. *μJava: An Automated Class Mutation System.* STVR, 15(2), 2005.

13. M. Chen et al. *Evaluating Large Language Models Trained on Code (Codex).* arXiv:2107.03374, 2021.

14. Y. Li et al. *Competition-Level Code Generation with AlphaCode.* arXiv:2203.07814, 2022.

15. C. E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.

16. J. Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.* arXiv:2405.15793, 2024.

17. S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models.* arXiv:2210.03629, 2022.

18. N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning.* arXiv:2303.11366, 2023.

19. A. Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback.* arXiv:2303.17651, 2023.

20. G. Irving, P. Christiano, D. Amodei. *AI Safety via Debate.* arXiv:1805.00899, 2018.

21. Y. Bai et al. *Constitutional AI: Harmlessness from AI Feedback.* arXiv:2212.08073, 2022.

## 9 9. Conclusion and Future Work

We presented Progee v2 as an evidence-first, state-machine-governed system for AI-assisted software engineering, and provided a reproducible evaluation path via E2E tests with XML outputs.

Future work:

- complete and validate a minimal mutation testing executor for Delphi/DUnitX workflows
- add standardized DB export scripts for evaluation metrics

## 10 Artifact Availability

- Replication guide: `docs/Replication.md`
- One-command E2E runner: `scripts/run_e2e_tests.ps1`
- DUnitX XML output (generated): `bin/dunitx-results.xml`